

Reflection Report

Challenges in Optimizing DOM Operations

One of the main challenges encountered in benchmarking the four JavaScript frameworks was preventing unnecessary rerendering of the entire task list.

In React, using unstable or missing keys when rendering lists caused the reconciliation algorithm to recreate every task whenever a single item changed. Assigning a unique task ID as the key solved this problem by allowing React to identify only the modified elements.

In Angular, optimization required additional configuration. Without using the `trackBy` function in `*ngFor`, Angular recreated all DOM elements during updates. Additionally, Angular's `zone.js` change detection could trigger extra rendering cycles unless updates were grouped into a single operation.

By comparison, Vue and Svelte required much less optimization. Vue's reactive system automatically updates only the affected components, while Svelte generates optimized JavaScript during compilation, resulting in direct DOM updates with minimal overhead.

Effect of Each Framework's Update Model on Performance

Each framework manages DOM updates differently which directly affects its performance.

- React uses a Virtual DOM to compare the previous and current UI before applying changes to the real DOM.
- Angular relies on its change detection mechanism, powered by `zone.js`, to determine which parts of the application need updating.
- Vue uses a proxy-based reactivity system that tracks dependencies and updates only the necessary elements.
- Svelte performs most of its optimization during compilation, producing JavaScript that directly modifies the required DOM nodes without using a Virtual DOM.

Since Vue and Svelte know exactly which elements need to change, they maintain excellent performance even when the number of tasks increases. React and Angular introduce additional processing due to Virtual DOM reconciliation and change detection, making updates slightly slower for larger lists.

Best Performance by Scenario

Based on the benchmark results:

- Svelte achieved the best overall performance especially during update and delete operations. Its compile time optimization enables direct DOM manipulation without runtime diffing.
- Vue delivered performance very close to Svelte and showed excellent efficiency during large rendering operations.
- React produced solid rendering performance but experienced additional overhead during updates because of the Virtual DOM reconciliation process.
- Angular performed sufficiently but recorded the highest execution times due to its comprehensive change detection mechanism.

Overall, Svelte was the fastest framework across most benchmark scenarios, followed closely by Vue and React remained competitive while offering a mature ecosystem while Angular even though it was slower in these tests but still provides a robust framework with many built-in features that are valuable for large enterprise applications.